

Module: Les technologies du Middleware



TP N 2: *Middleware* à objets répartis : java RMI (*Java RemoteMethod Invocation*)



- Java RMI: Présentation.
- Java RMI: Architecture.
- Java RMI: Règles d'usage.
- Java RMI: Exemple



Java RMI: Présentation

- RMI est un système d'objets distribués constitué uniquement d'objets java ;
- RMI est une API (Application Programming Interface) intégrée au JDK 1.1 et plus ;
- Développé par JavaSoft ;
- Mécanisme qui permet l'appel de méthodes entre objets Java qui s'exécutent éventuellement sur des JVM distinctes ;
- L'appel peut se faire sur la même machine ou bien sur des machines connectées sur un réseau ;
- Utilise les sockets ;
- Les échanges respectent un protocole propriétaire : **Remote Method Protocol** ;
- RMI repose sur les classes de sérialisation.

Java RMI (Remote Method Invocation)



■ Permet la communication entre machines virtuelles Java (JVM) qui peuvent se trouver physiquement sur la même machine ou sur deux machines distinctes.

Motivation : construction d'applications réparties avec Java

Appel de méthode au lieu d'appel de procédure

Principe

Même schéma que RPC

Le programmeur fournit une (ou plusieurs) description(s) d'interface

Ici pas d'**IDL (Interface Development Language)** séparé : Java sert d'IDL (les interfaces des objets distants)

Applications **Java à Java** uniquement



Notion d'IDL

IDL (Interface Development Language)

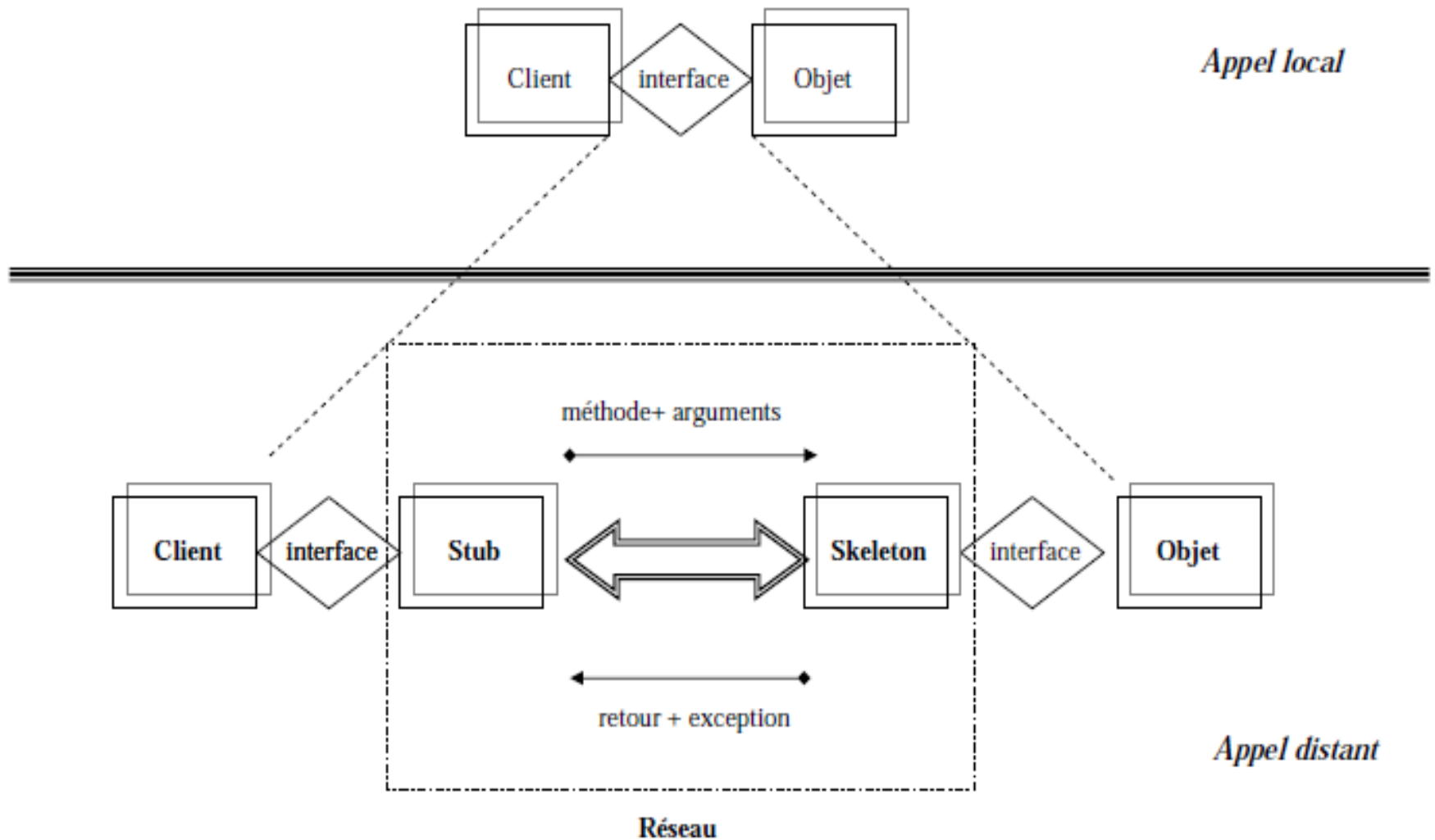
est un langage voué à la définition de l'interface de composants logiciels, laquelle permet de faire communiquer des modules implémentés dans des langages différents, ou déployés à travers un réseau sur des systèmes hétérogènes (Windows, Linux, Mac OS X, VMS, etc.) dans la perspective d'architecture distribuées.

En Java RMI, Les interfaces des objets distant servent d'IDL.



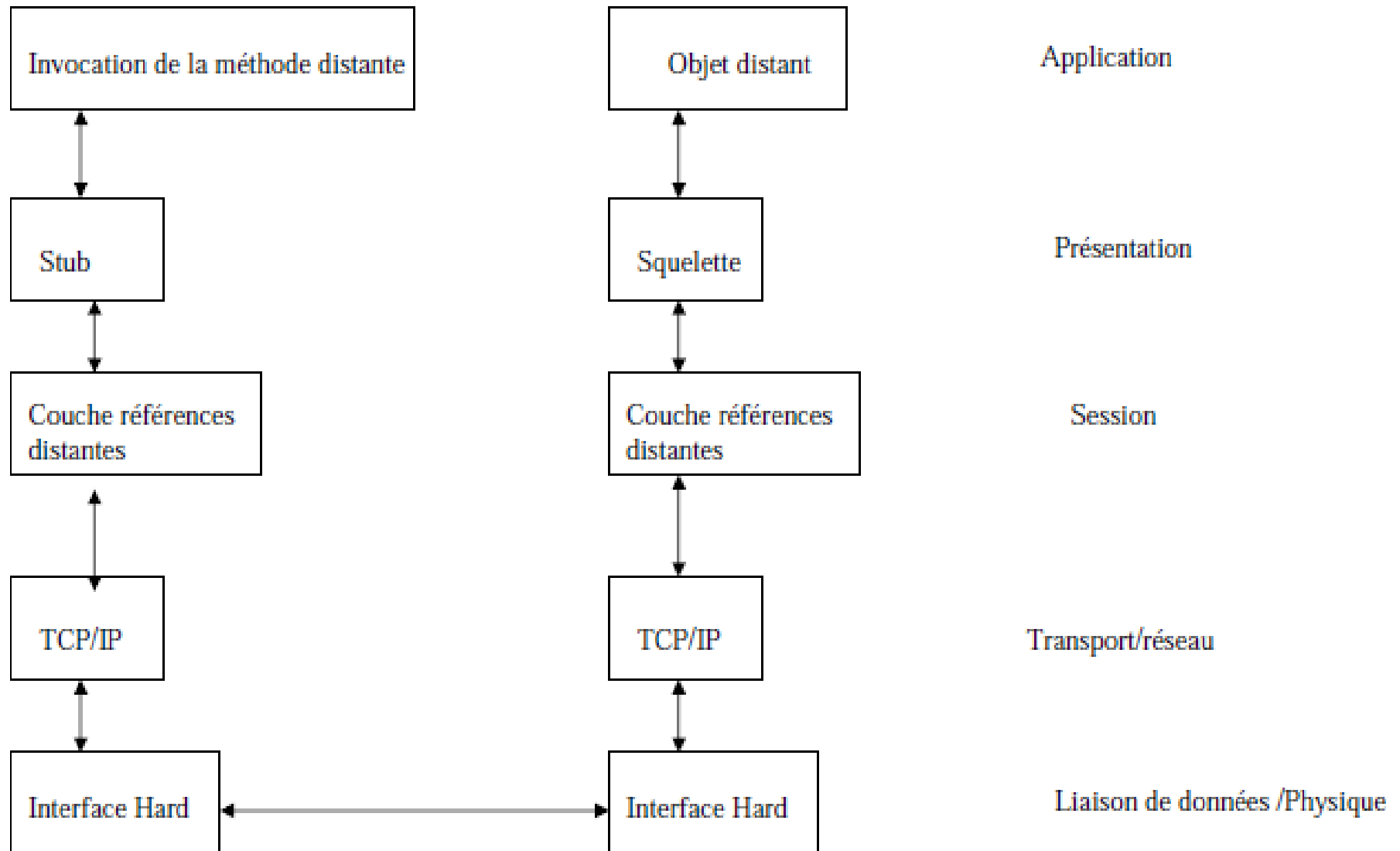
JAVA RMI : Appel local versus Appel distant

Un objet sur une JVM peut avoir une référence sur un autre dans une autre JVM



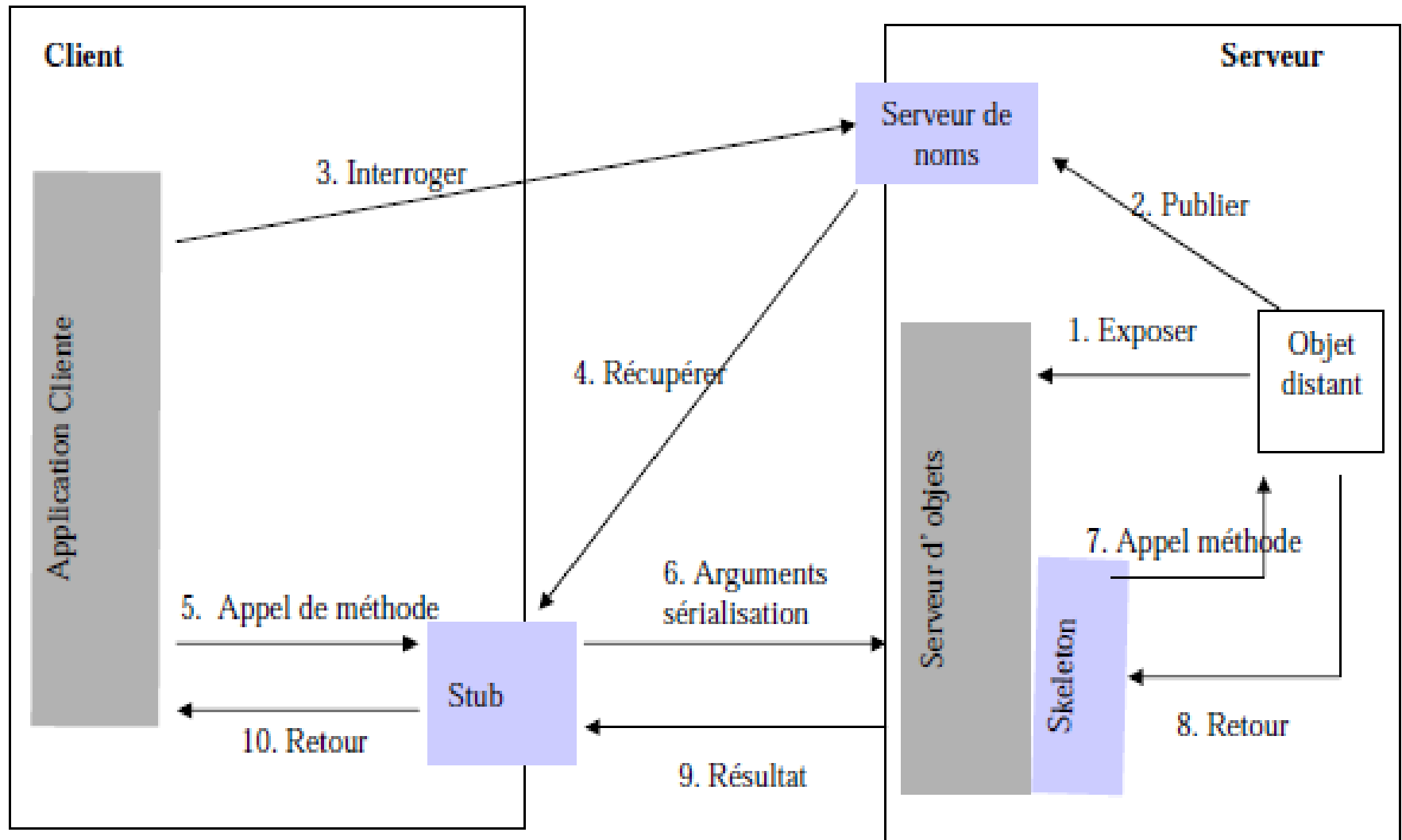


JAVA RMI : Appel d'une méthode d'un objet distance



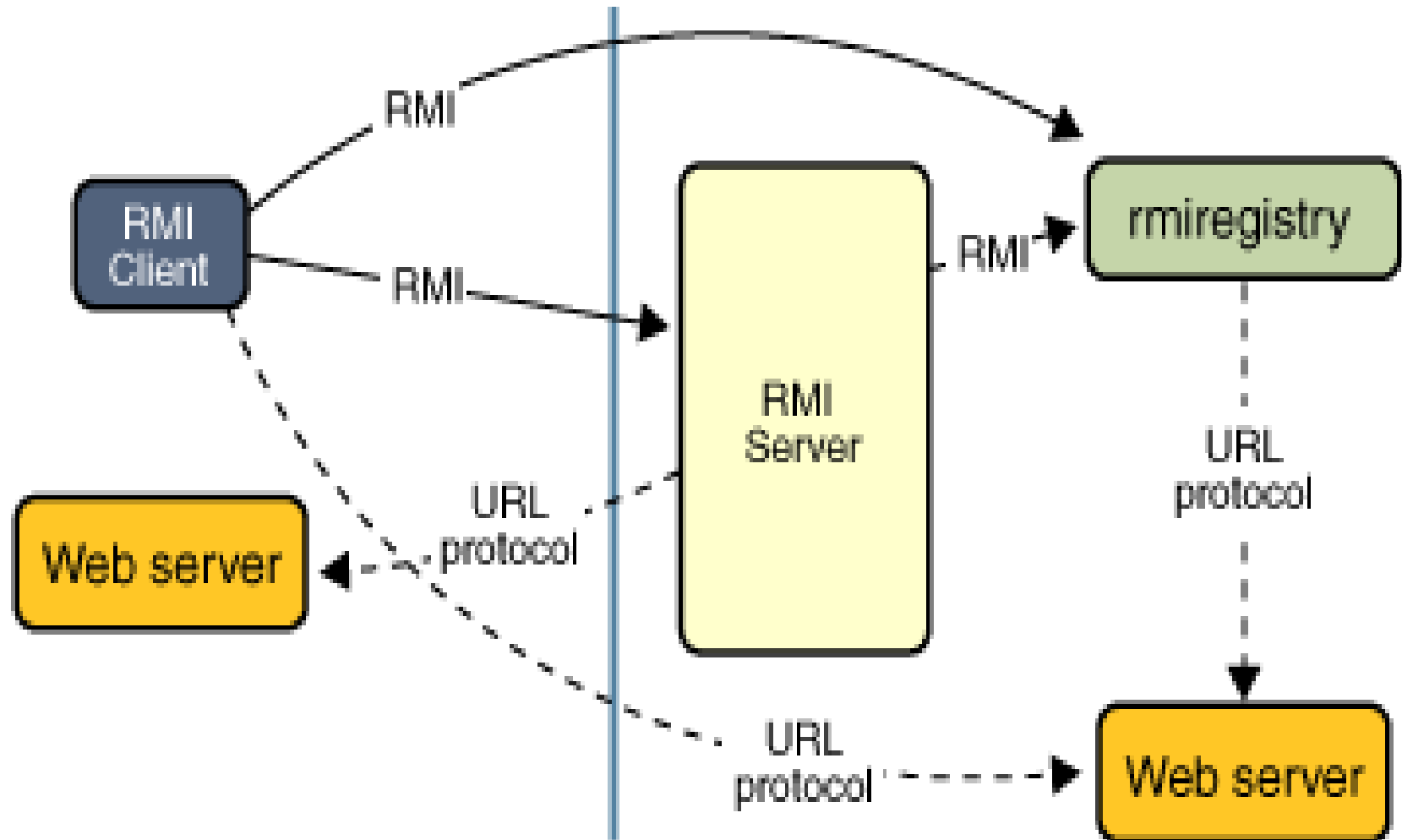


Etapes d'invocation d'une méthode d'un objet distant





Java RMI: Architecture





Elles assurent le rôle d'adaptateurs pour le transport des appels distants

- ✓ Elles réalisent les appels sur la couche réseau
- ✓ Elles réalisent l'assemblage et le désassemblage des paramètres (*marshalling, unmarshalling*)
- ✓ Une référence d'objets distribué correspond à une référence d'amorce
- ✓ Les amorces sont créées par le générateur **rmic** (pas besoin de générer les amorces JDK 1.5 ++).



Représentants locaux de l'objet distribué (coté client)

- ✓ Initient une connexion avec la JVM distante
- ✓ l'invocation distante à la couche des références d'objets ;
- ✓ Assemblent les paramètres pour leur transfert à la JVM distante
- ✓ Attendent les résultats de l'invocation distante ;
- ✓ Désassemblent la valeur ou l'exception renvoyée ;
- ✓ Renvoient la valeur à l'appelant ;
- ✓ S'appuient sur la **sérialisation**.



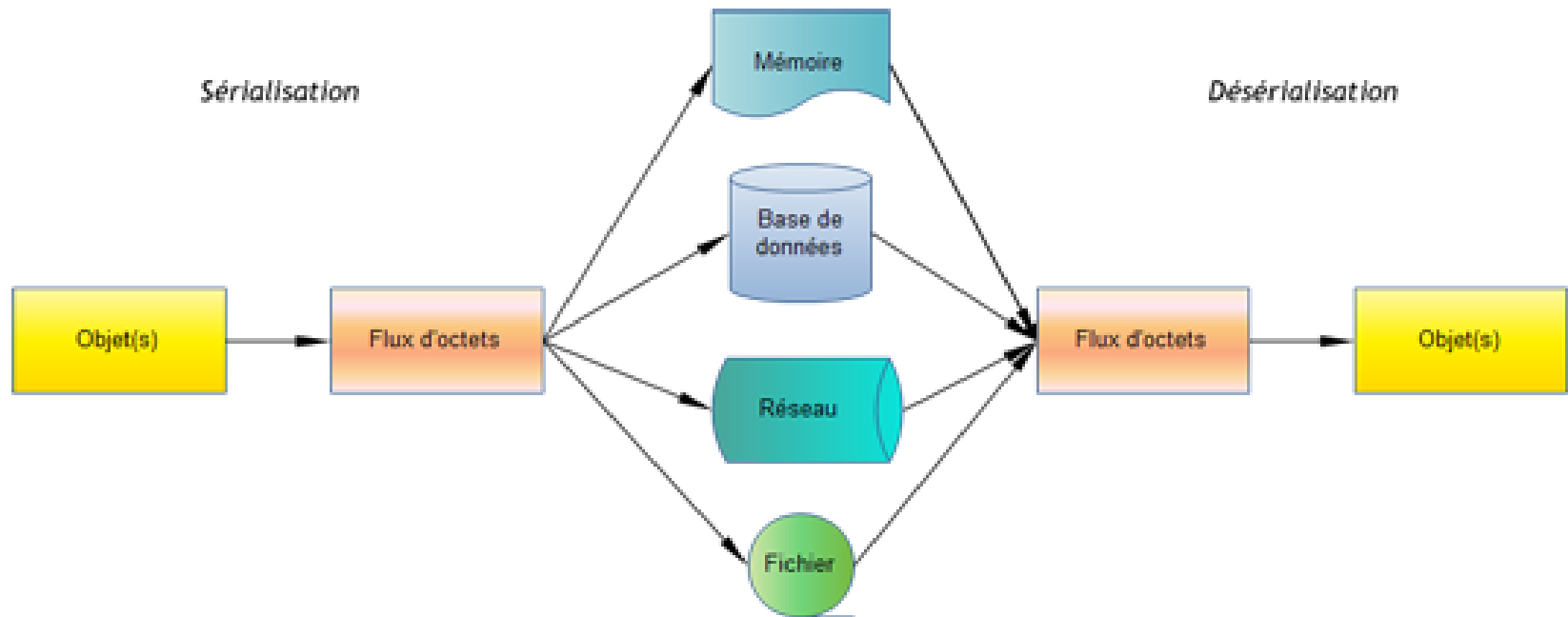
Coté Serveur (Stub serveur=Skeleton)

- ✓ Désassemblent les paramètres pour la méthode distante ;
- ✓ Font appel à la méthode demandée ;
- ✓ Assemblage du résultat (valeur renvoyée ou exception) à destination de l'appelant.
- ✓ S'appuient sur la **sérialisation**.



Passage d'objets en paramètres

- Un objet sérialisable (transmissible par valeur hors de sa JVM) doit implémenter `java.io.Serializable`
- Les objets référencés dans un objet sérialisable doivent aussi être sérialisables





RMI: Notions sur les objets sérialisables (2)

Comment rendre effectivement un objet sérialisable ?

- Pour les variables de types primitifs (int, boolean,..) rien à faire.
- Pour les objet dont les champs sont constitués de telles variables : rien à faire.
- Pour un champ non immédiatement sérialisable (ex. : Array), il faut fournir des méthodes `readObject()` et `writeObject()`

Voir javadoc de l'interface `Serializable` et des classes
`ObjectOutputStream` et `ObjectInputStream`



Permet d'obtenir une référence d'objet distribué à partir de la référence locale au stub

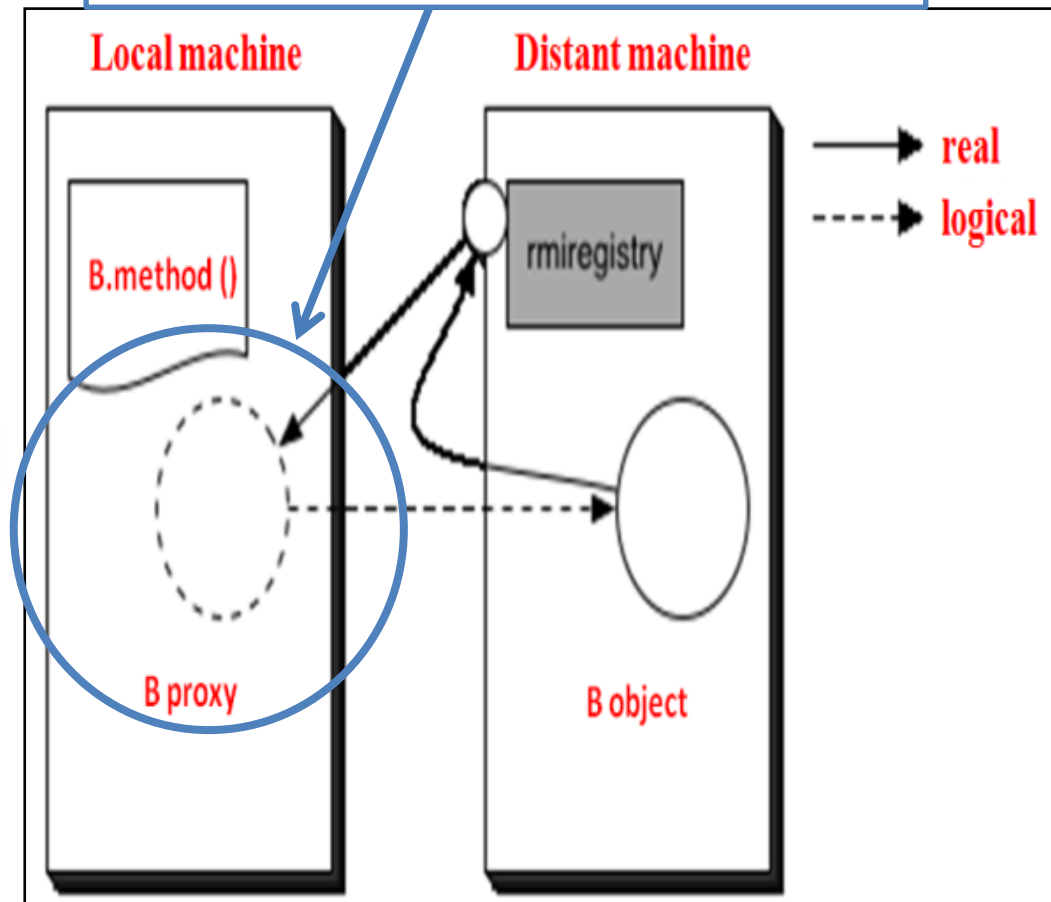
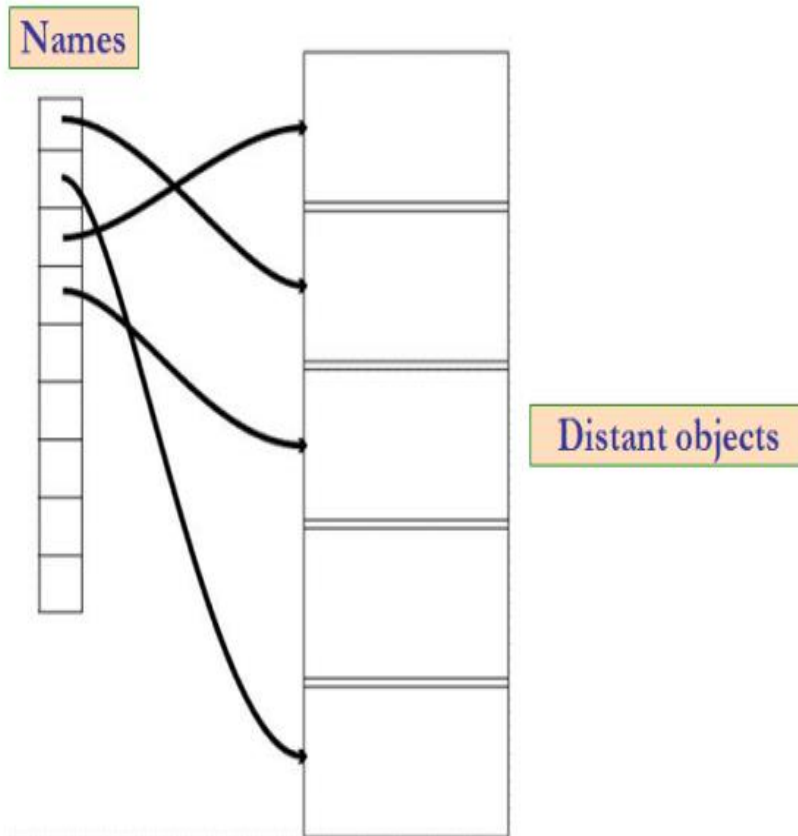
- Cette fonction est assurée grâce à un service de noms **rmiregistry**
- Un unique **rmiregistry** par JVM ;
- **Rmiregistry** s'exécute sur chaque machine hébergeant des objets distants ;
- **Rmiregistry** accepte des demandes de service sur le port 1099;

RMI: *rmiregistry*



Rmiregistry possède une table de hachage dont les clés sont des noms et les valeurs sont des objets distants

B proxy est le stub client qui a une référence vers l'objet distant et communique avec le stub serveur (sekelton)



Rmiregistry



Réalise les connexions réseau basées sur les flux entre les JVM

- Emploie un protocole de communication propriétaire (**JRMP**: Java Remote Method Invocation) basé sur TCP/IP
- Le protocole JRMP a été modifié afin de supprimer la nécessité des squelettes car depuis la version 1.2 de Java, une même classe skeleton générique est partagée par tous les objets distants.



➤ Le programme du serveur

- Objets réalisant les interfaces

➤ Le programme du client

- L'environnement Java fournit
- Un générateur de talons (**rmic**)
- Un service de noms (Object Registry)



➤ Interface (IDL)

L'interface d'un objet distant (Remote) est celle d'un objet Java, avec quelques règles d'usage :

- ✓ L'interface **distante** doit être **publique**
- ✓ L'interface distante doit étendre l'interface **java.rmi.Remote**
- ✓ Chaque méthode doit déclarer au moins l'exception **java.rmi.RemoteException**



Le compilateur IDL : **rmic**

- Le compilateur **rmic** génère les amorces (stub/skeleton)
- Une référence d'objets distant correspond à une référence d'amorce (stub).



Passage d'objets en paramètres

- ✓ Les objets locaux sont passés par valeur (copie) et doivent être **sérialisables** (étendent l'interface **java.io.Serializable**)
- ✓ Les objets distants sont passés par référence et sont désignés par leur interface

Réalisation des classes distantes (Remote)

- ✓ Une classe distante doit implémenter une interface elle-même distante (Remote)
- ✓ Une classe distante doit étendre la classe **java.rmi.server.UnicastRemoteObject** (d'autres possibilités existent)
- ✓ Une classe distante peut aussi avoir des méthodes appelables seulement localement (ne font pas partie de son interface Remote)



Un serveur est une classe qui implémente l'interface de l'objet distant
Spécifier les références distantes qui doivent être implémentées (objets passées en paramètres)

- ✓ Définir le constructeur de l'objet distant.
- ✓ Fournir la réalisation des méthodes appelables à distance.
- ✓ Créer et installer le gestionnaire de sécurité.
- ✓ Créer au moins une instance de la classe serveur.
- ✓ Enregistrer au moins une instance dans le serveur de noms.

Java RMI : Règles d'écriture du Client



- ✓ Pour se connecter au serveur le client doit récupérer une instance stub.
- ✓ Le stub déléguer l'appel de la méthode au travers d'un socket.

Java RMI : Etapes de développement (1)



Interfaces distantes

- ✓ Choisir les informations que l'on doit accéder à distance
- ✓ Les structurer en objet: interface sans accès direct à des données (uniquement des routines)
- ✓ Définir l'interface avec les méthodes et les exceptions (RemoteException + éventuellement d'autres)

Serveur RMI

- ✓ Définir des classes qui implémentent les Interfaces **Distantes**
UnicastRemoteObject (en général)

Enregistrement d'au moins un objet dans le **rmiregistry**

Java RMI : Etapes de développement (2)



- ✓ Déclaration de références (attribues, etc.) vers des Interfaces Java Distantes (Remote)
- ✓ Récupération d'au moins un objet par le **rmiregistry**
- ✓ Appels de méthodes classiques, avec récupération des exceptions

Compiler les sources

- ✓ Clients et serveurs: **javac**

Précompilation: création Stubs/Skeletons

- ✓ Utilisation de **rmic** pour générer les Stub/Skeletons pour toutes les classes qui implémentent Remote.
- ✓ Il faut, auparavant, que les classes “Remote” compilées (javac).

Java RMI : Etapes de développement (3)



Lancer le rmiregistry

- ✓ Choisir un port (défaut: 1099)
- ✓ Lancer le **rmiregistry**

Installer le serveur

- ✓ Lancer un ou plusieurs serveurs

Lancer le ou les clients

- ✓ Lancer un ou plusieurs clients
- ✓ On indiquera en général au client, en paramètres de la ligne de commande :
 - où (host), et
 - sur quel port

le serveur écoute ...

Java RMI : Exemple Hello (1)



Remote Interface

```
public interface Hello extends java.rmi.Remote
{
String sayHello() throws java.rmi.RemoteException;
}
```

Serveur HelloWorld

```
import java.net.InetAddress;
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.RMISecurityManager;
import java.rmi.server.UnicastRemoteObject;
public class HelloServer
extends UnicastRemoteObject
implements Hello
{
public HelloServer() throws RemoteException { }
```

Java RMI : Exemple Hello (2)



```
public String sayHello()
{
    return "Hello World from " + getHostName();
}
protected static String getHostName()
{
    try
    {
        return InetAddress.getLocalHost().getHostName();
    }
    catch (java.net.UnknownHostException who)
    {
        return "Unknown";
    }
}
```

Java RMI : Exemple Hello (3)



Enregistrement dans le rmiregistry

Création de l'objet accessible à distance, et enregistrement dans le rmiregistry. (**HelloServerMain.java**).

```
public static void main(String args[])  
{  
    // Create and install a security manager  
    System.setSecurityManager(new RMISecurityManager());  
    try  
    {  
        // Create and register the server object  
        HelloServer serverObject = new HelloServer();  
        Naming.rebind// Objet Remote ...  
        // not the remote interface: Hello  
        // not the stub  
        // How does it work ???  
        nd("rmi://RMIServerHostName/HelloServer", serverObject );
```

Java RMI : Exemple Hello (4)



```
// Signal successful registration
System.out.println("HelloServer bound in registry");
}
catch (Exception e)
{
System.out.println("HelloServer err: ");
e.printStackTrace();
}}}
```

Naming :

```
Naming.rebind("rmi://RMIHostName/HelloServer", serverObject
);
```

Java RMI : Exemple Hello (5)



La syntaxe complète pour Naming :

voir(<https://docs.oracle.com/javase/7/docs/api/java/rmi/Naming.html>)

rebind et lookup est la suivante:

"rmi://host:port/ServerLabel"

“rmi:“, machine, et “port” (si 1099) sont optionnels dans le cas d’un bind et rebind :

```
Naming.rebind( "rmi://RMIServerHostName:1099/HelloServer", serverObject);
```

ou

```
Naming.rebind( "///RMIServerHostName/serverObject);
```

ou

```
Naming.rebind( "HelloServer", serverObject);
```

Les méthodes retournent void et throw:

- MalformedURLException (not an URL)
- RemoteException (registry contact failed)
- AccessException (non-local host)

Java RMI : Exemple Hello (6)



Par contre on indiquera toujours au moins la machine pour un lookup:

```
Hello remote = (Hello) Naming.lookup( "//RMIServerHostName:1099/HelloServer");
```

```
Hello remote = (Hello) Naming.lookup( "//RMIServerHostName/HelloServer");
```

sachant que si le port n'est pas spécifié, on utilise celui par défaut :
1099

On ne peut bind ou unbind sur un rmiregistry, que depuis une VM sur le même host (sécurité).

lookup depuis n'importe quel host.

Java RMI : Exemple Hello (7)



Un client de HelloWorld

```
import java.rmi.*;
import java.net.MalformedURLException;
public class HelloClient
{
    public static void main(String args[])
    {
        try {
            Hello remote = (Hello) Naming.lookup(
                "rmi://RMIServerHostName/HelloServer");
            String message = remote.sayHello();
            System.out.println( message );
        }
        catch ( NotBoundException error)
        {
            error.printStackTrace();
        }
    }
}
```

Java RMI : Exemple Hello (8)



```
catch ( MalformedURLException error)
{
    error.printStackTrace();
}
catch ( UnknownHostException error)
{
    error.printStackTrace();
}
catch ( RemoteException error)
{
    error.printStackTrace();
}
}
```

Les catch multiples permettent de détecter quelle exception est levée.

Java RMI : Exemple (9)



Compilation, génération, registry

Côté Serveur

Pour Java 2 et plus, placer le fichier suivant dans votre home directory:
.java.policy

```
grant {  
  // Allow everything for now  
  permission java.security.AllPermission;  
};  
// Ne pas utiliser ce fichier systématiquement !!
```

Compiler les sources:

```
javac Hello.java HelloServer .java
```

```
** Hello.java **
```

lien, connaissance statique Client/serveur*

Java RMI : Exemple Hello (10)



Générer les Stubs et les Skeletons

```
rmic HelloServer
```

Produit:

```
HelloServer_Skel.class
```

```
HelloServer_Stub.class
```

```
Démarrer le rmiregistry
```

```
rmiregistry 2001 &
```

Démarrer le serveur:

```
java HelloServer
```

Java RMI : Exemple Hello (11)



Côté Client

Compiler les sources:

```
javac Hello.java HelloClient.java
```

On a besoin de l'interface remote (Hello.java)

```
** Hello.java **
```

lien, connaissance statique Client/serveur*

Rendre la classe HelloServer_Stub.class disponible du côté du client :

Copier le fichier HelloServer_Stub.class du serveur à la machine client (ou NFS)

Lancer le client:

```
java HelloClient
```

La place du serveur est dans le code, mais normalement, il faut utiliser un argument de la ligne de commande.



Notez bien:

Avec Java RMI, on peut créer en local des objets accessibles à distance

mais on ne peut pas directement:

créer directement à distance des objets accessibles à distance !

On ne peut pas du tout:

Créer directement à distance une JVM sur une machine pour laquelle on n'a pas déjà une JVM avec un objet remote accessible:

- Enregistré dans le registry,
- Ou transitivement connu d'un autre objet remote, ... lui même connu.

Java RMI : Security Manager (1)



Sécurité et chargement dynamique de code

Un **codebase** est un endroit depuis lequel charger du code dans une VM.

java.rmi.server.codebase : spécifie l'URL (file://, ftp://, http://) où peuvent se trouver les classes. Lorsque RMI sérialise l'objet (envoyé comme paramètre ou reçu comme résultat), il rajoute l'URL spécifiée par codebase.

- **CLASSPATH** est un **codebase** local

- autre exemple de codebase:

<http://myhost/~myusername/myclasses/>

- Le **codebase** d'une applet est toujours relatif à l'URL depuis laquelle elle a été chargée.

java.rmi.server.useCodebaseOnly : informe le client que le chargement de classes est fait uniquement à partir du répertoire du **codebase**

Java RMI : Security Manager (2)



Security Manager

Toute JVM qui doit downloader du code a besoin d'un Security Manager (SM).

<https://docs.oracle.com/javase/tutorial/essential/environment/security.html>

Principe d'un Security Manager:

- si aucun security manager n'est installé, alors seules les classes accessibles depuis le **CLASSPATH** peuvent être chargées.
- un security manager vérifie différents critères sur les classes au chargement.

Java RMI : Security Manager (3)



Une façon de charger un `SecurityManager` uniquement s'il n'y en a pas déjà un :

```
if (System.getSecurityManager() == null)
{
    System.setSecurityManager(
        newRMISecurityManager());
}
```

Voir

<https://docs.oracle.com/javase/7/docs/api/java/lang/SecurityManager.html>

Un **SecurityManager** doit être utilisé avec RMI, sinon les clients et les serveurs RMI ne peuvent pas charger de code autre que directement par le CLASSPATH.

Java RMI : chargement dynamique du code



- A l'enregistrement (dans `rmiregistry`) de l'objet distant, le codebase est spécifié par `java.rmi.server.codebase`.
- A l'appel de `bind()`, le registre utilise ce codebase pour trouver les fichiers de classe associés à l'objet.
- Le client recherche la définition de classe du stub dans son classpath. S'il ne la trouve pas, il essaiera de la récupérer à partir du codebase.
- Une fois que toutes les définitions de classe sont disponibles, la méthode proxy du stub appelle les objets sur le serveur.
- Les classes `java.rmi.RMISecurityManager` et `java.rmi.server.RMIClassLoader` vérifient le contexte de sécurité avant de charger des classes à partir d'emplacements distants.
- La méthode **`LoadClass`** de **`RMIClassLoader`** charge la classe à partir du codebase spécifié. **Sécurité lors d'un chargement .**

Java RMI : Chargement dynamique du code



- Écrire les classes correspondant respectivement à l'interface et à l'objet.
- Les compiler.
- Générer le *Stub* correspondant à l'objet.
- Installer tous les fichiers de classe sur un serveur Web.
- Écrire le serveur dynamique.
- Installer *rmiregistry* au niveau de la machine du serveur.
- Lancer le serveur en lui précisant l'URL des fichiers de classe afin qu'il puisse charger dynamiquement le fichier de classe correspondant à l'objet, l'instancier (le créer) et l'enregistrer auprès de *rmiregistry*.
- Sur la machine du client, écrire le code du client.
- Compiler le client statique et l'installer éventuellement sur le site Web.
- Compiler le client dynamique et le lancer en précisant l'URL des fichiers de classe.



1. Pitt, Kathleen McNiff, java(TM).rmi The Remote Method Invocation Guide by Esmond .
2. Badr Benmammar , **Concurrent, Real-Time and Distributed Programming in Java, FOCUS SERIES** Jean-Charles Pomerol (Wiley), 2018.
3. Samia Bouzefrane, Cours : ***Intergiciels à Objets Répartis : Java RemoteMethod Invocation (RMI) ou les invocations de méthodes Java distantes.***
4. Support de cours Environnement de distribution et architecture répartie, Université de la Réunion, inspiré du Cours de S. Krakowiak.
5. J. Christian Attiogbe, Remote Method Invocation en Java (RMI) Modelisation et construction des applications réparties. 2015
6. <https://docs.oracle.com/javase/tutorial/rmi/overview.html>
7. [Denis Caromel](#), [Brian Amedro](#), TD de module **Programmation Répartie et Architecture N 1/3**, Université Sophia Anthipolis, 2013.